C_Pointers

Task 2

Embedded SW Committee

IEEE SSCS AUSC

March 1, 2026

Task 1: ADC Buffer & Signal Processing

1) Background

In embedded systems, sensors are read using an **ADC (Analog to Digital Converter)**.

The ADC stores readings in a **buffer (array)**.

These values are processed using pointer-based functions for efficiency.

This simulates real firmware where data is passed using pointers instead of copying arrays.

Task 1: ADC Buffer & Signal Processing

2) Objective
Implement a system that processes ADC readings using **pointers** to:

Calculate statistics
Normalize signal
Detect threshold crossing

Task 1: ADC Buffer & Signal Processing

3) Requirements

Given buffer:

uint16_t adc_buffer[10] = {120, 340, 560, 230, 800, 1023, 450, 300, 700, 200};

Implement these functions:

1) getMax
2) getMin
3) getAverage
4) Normalize
5) detectThreshold

Task 1: ADC Buffer & Signal Processing

4) Behavior

getMax → return maximum value
getMin → return minimum value
getAverage → return average
normalize → scale values to range (0 → 1)
detectThreshold → return 1 if any value exceeds threshold

Task 2: Automotive Firmware Task

1) Background

You are a Firmware Engineer working on an **Electronic Brake Control Unit (EBCU)** in an automotive safety system.

The module is responsible for:

- Reading wheel speed sensors.
- Maintaining rolling buffers of measurements.
- Computing filtered values.
- Calculating slip ratio.
- Controlling brake pressure via PWM.

The system passed initial bench testing and vehicle-level validation.

Task 2: Automotive Firmware Task

1) Background

The system:

- The ECU stored wheel speed samples in a fixed-size buffer:
  - `#define BUF_SIZE 8`
  - `int wheel_speed_buf[BUF_SIZE];`
- Reading sensor were stored using pointer
  - `int *write_ptr = wheel_speed_buf;`
- Inserted values using pointer arithmetic
- Performed moving average filtering
- Calculated delta between samples
- Adjusted braking force accordingly

All pointer operations were deterministic.

Vehicle braking was smooth and stable.

Task 2: Automotive Firmware Task

1) Background

A firmware update:

Support both integer and floating-point sensor formats using generic processing

<u>The results after first deployment:</u>

- Sudden brake pulsation occurred
- ABS activated unexpectedly
- Fault flags were intermittently triggered
- Emergency braking showed unstable modulation

The issue was traced to memory corruption and pointer misuse.

Task 2: Automotive Firmware Task

1) Background

A firmware update:

Support both integer and floating-point sensor formats using generic processing

The instability was software-induced and directly related to improper pointer handling.

- Unsafe void* casting
- Incorrect pointer arithmetic
- Misuse of pointer-to-array
- Missing NULL validation
- Returning address of local variable (dangling pointer)
- Usage of uninitialized pointer (wild pointer)

Task 2: Automotive Firmware Task

2) Objective

Redesign and validate the pointer-based processing logic to ensure:

- Deterministic memory behavior
- Safe pointer arithmetic
- Correct pointer-to-array usage
- Defensive NULL handling
- Elimination of wild and dangling pointers
- Compliance with safety-oriented embedded coding practices

Task 2: Automotive Firmware Task

3) Requirements

1. Circular Buffer Implementation

The system shall implement a fixed-size circular buffer for wheel speed samples using pointer-based logic.

Requirements:

1.1 The buffer shall be defined as a statically allocated array.

1.2 Data insertion shall be performed using pointer arithmetic only.

1.3 Array indexing ([ ]) shall not be used inside processing functions.

1.4 The write pointer shall increment deterministically and wrap to the base address when reaching the buffer boundary.

1.5 The implementation shall ensure no memory access beyond buffer limits.

1.6 All pointer operations shall be deterministic and free from undefined behavior.

Task 2: Automotive Firmware Task

3) Requirements

2. Pointer Arithmetic Validation

The system shall perform data processing using pointer traversal.

Requirements:

2.1 Sum calculation shall be implemented using pointer increment operations.

2.2 Moving average computation shall use pointer traversal without array indexing.

2.3 Delta calculation shall correctly reference the last two valid buffer elements using pointer arithmetic.

2.4 All pointer arithmetic shall be based on correct type size interpretation.

2.5 No out-of-bounds access shall occur under any operating condition.

Task 2: Automotive Firmware Task

3) Requirements

3. Pointer-to-Array Handling

The system shall demonstrate correct usage of pointer-to-array types.

Requirements:

3.1 A pointer-to-array shall be declared and initialized correctly.

3.2 The implementation shall compare address increments between:

Pointer to element

Pointer to array

3.3 The system shall document and explain the difference in memory offset behavior.

3.4 No misuse of pointer-to-array arithmetic shall be permitted.

Task 2: Automotive Firmware Task

3) Requirements

4. Generic Processing Using void Pointer

The system shall support generic buffer processing using void*.

Requirements:

4.1 The generic processing function shall accept:
A pointer to data buffer
Element count
Element size

4.2 The implementation shall perform pointer arithmetic using byte-wise traversal (unsigned char*).

4.3 The system shall avoid unsafe casting that may cause alignment faults.

4.4 The implementation shall not violate strict aliasing rules.

4.5 The function shall validate NULL input before processing.

4.6 The solution shall be portable across 32-bit and 64-bit architectures.

Task 2: Automotive Firmware Task

3) Requirements

5. Pointer Safety Validation

The system shall explicitly demonstrate safe handling of pointer edge cases.

5.1 NULL Pointer Handling

5.1.1 All dereference operations shall validate pointer integrity before access.

5.1.2 The system shall return status codes instead of triggering runtime faults.

5.2 Wild Pointer Demonstration

5.2.1 An uninitialized pointer scenario shall be declared.

5.2.2 The pointer shall not be dereferenced.

5.2.3 The implementation shall document associated safety risks.

5.3 Dangling Pointer Scenario

5.3.1 A function returning the address of a local variable shall be implemented for demonstration.

5.3.2 The returned pointer shall not be dereferenced.

5.3.3 The lifetime violation shall be documented and explained.

Thank You!

Questions?